

LIMBAJE FORMALE ȘI AUTOMATE - LABORATOR 6

MARIA PREDA

În cadrul acestui laborator vom continua să vorbim despre gramatici independente de context și doi algoritmi fundamentali care le folosesc: **transformarea în Forma Normală Chomsky (CNF)** și **algoritmul CYK** de recunoaștere a cuvintelor. Vom încheia cu câteva explicații suplimentare despre **parsarea LR(1)**, implementată în cadrul laboratorului trecut, utilizată în compilatoare.

1. BREVIAR TEORETIC

1.1. Forma Normală Chomsky.

Definiție 1.1 (Forma Normală Chomsky). *O gramatică $G = (V, \Sigma, S, P)$ este în Forma Normală Chomsky (CNF) dacă fiecare producție are una dintre formele:*

$$A \rightarrow BC \quad \text{sau} \quad A \rightarrow a$$

unde $A, B, C \in V$ și $a \in \Sigma$.

Se admite și producția $S \rightarrow \lambda$ dacă $\lambda \in L(G)$.

Teoremă 1.2. *Orice limbaj independent de context poate fi generat de o gramatică în Forma Normală Chomsky.*

1.1.1. *Intuiție.* Forma Normală Chomsky impune ca fiecare producție să fie foarte simplă: fie produce exact două neterminale, fie produce exact un terminal.

Această restricție este utilă deoarece orice derivare poate fi privită ca un arbore binar. Din acest motiv, CNF este foarte potrivită pentru algoritmi de parsare bazați pe programare dinamică, cum este algoritmul CYK.

1.1.2. *Algoritmul de transformare în CNF.* Transformarea unei gramatici în CNF se face în mai mulți pași:

- (1) adăugarea unui nou simbol de start (dacă este necesar)
- (2) eliminarea producțiilor λ
- (3) eliminarea producțiilor unitare
- (4) eliminarea simbolurilor inutile
- (5) binarizarea producțiilor lungi
- (6) eliminarea terminalelor din producțiile binare

Pasul 2: Eliminarea producțiilor λ .

Determinăm mulțimea neterminalelor anulabile:

$$\text{Null} = \{A \in V \mid A \Rightarrow^* \lambda\}.$$

Pentru fiecare producție care conține simboluri anulabile, adăugăm variantele obținute prin eliminarea acestor simboluri. La final, eliminăm producțiile de forma $A \rightarrow \lambda$, cu excepția cazului în care A este simbolul de start și λ aparține limbajului.

Pasul 3: Eliminarea producțiilor unitare.

O producție unitară este o producție de forma:

$$A \rightarrow B,$$

unde A și B sunt neterminale.

Pentru fiecare astfel de relație, transferăm producțiile neterminalului B către neterminalul A , apoi eliminăm producția unitară.

Pasul 4: Eliminarea simbolurilor inutile.

Eliminăm:

- simbolurile care nu pot genera niciun cuvânt format doar din terminale;
- simbolurile nu sunt accesibile pornind din simbolul de start.

Pasul 5: Binarizarea producțiilor lungi.

Pentru o producție de forma:

$$A \rightarrow X_1 X_2 \cdots X_k, \quad k \geq 3,$$

introducem neterminale noi și o înlocuim cu producții binare:

$$A \rightarrow X_1 N_1, \quad N_1 \rightarrow X_2 N_2, \quad \dots, \quad N_{k-2} \rightarrow X_{k-1} X_k.$$

Pasul 6: Eliminarea terminalelor din producțiile binare.

Dacă apare o producție de forma:

$$A \rightarrow aB$$

sau

$$A \rightarrow Ba,$$

introducem un neterminal nou T_a cu regula:

$$T_a \rightarrow a$$

și înlocuim terminalul a cu T_a .

Observație 1.3. Ordinea pașilor este importantă. De exemplu, eliminarea producțiilor λ poate introduce producții unitare noi.

Exemplu 1.4. Considerăm gramatica:

$$\begin{aligned} S &\rightarrow ASB \mid \lambda \\ A &\rightarrow aAS \mid a \mid \lambda \\ B &\rightarrow SbS \mid A \mid bb \end{aligned}$$

Neterminalele anulabile sunt:

$$Null = \{S, A\}.$$

Din producția $S \rightarrow ASB$ obținem, prin eliminarea aparițiilor anulabile ale lui A și S , producții precum:

$$S \rightarrow ASB \mid SB \mid AB \mid B.$$

Din producția $A \rightarrow aAS$ obținem:

$$A \rightarrow aAS \mid aS \mid aA \mid a.$$

După eliminarea producțiilor λ și a producțiilor unitare, producțiile rămase se binarizează. De exemplu:

$$S \rightarrow ASB$$

devine:

$$S \rightarrow AN_1, \quad N_1 \rightarrow SB.$$

Pentru terminalele care apar în producții lungi introducem:

$$T_a \rightarrow a, \quad T_b \rightarrow b.$$

1.2. Algoritmul CYK. Algoritmul *Cocke–Younger–Kasami* decide dacă un cuvânt aparține limbajului generat de o gramatică în CNF.

Complexitatea algoritmului este:

$$O(n^3 \cdot |G|),$$

unde n este lungimea cuvântului, iar $|G|$ este numărul de producții.

1.2.1. *Ideea de bază.* Fie:

$$w = a_1 a_2 \cdots a_n.$$

Construim un tabel T , unde:

$$T[i][j] = \{A \in V \mid A \Rightarrow^* a_i a_{i+1} \cdots a_j\}.$$

Cu alte cuvinte, $T[i][j]$ conține toate neterminalele care pot genera subșirul dintre pozițiile i și j .

Cuvântul este acceptat dacă:

$$S \in T[1][n].$$

1.2.2. *Recurența. Cazul de bază:*

$$T[i][i] = \{A \in V \mid A \rightarrow a_i \in P\}.$$

Cazul recursiv:

$$T[i][j] = \{A \in V \mid \exists k, A \rightarrow BC, B \in T[i][k], C \in T[k+1][j]\}.$$

Observație 1.5. Fiecare valoare k reprezintă un posibil punct de tăiere al subșirului $a_i \cdots a_j$ în două bucăți mai mici.

1.2.3. *Exemplu CYK.*

Exemplu 1.6. Fie gramatica în CNF:

$$S \rightarrow AB \mid BC$$

$$A \rightarrow BA \mid a$$

$$B \rightarrow CC \mid b$$

$$C \rightarrow AB \mid a$$

și cuvântul:

$$w = baaba.$$

Indexăm:

$$a_1 = b, \quad a_2 = a, \quad a_3 = a, \quad a_4 = b, \quad a_5 = a.$$

Diagonala 1:

$$T[1][1] = \{B\}$$

$$T[2][2] = \{A, C\}$$

$$T[3][3] = \{A, C\}$$

$$T[4][4] = \{B\}$$

$$T[5][5] = \{A, C\}$$

Diagonala 2:

$$T[1][2] = \{A, S\}$$

$$\text{deoarece } A \rightarrow BA, S \rightarrow BC$$

$$T[2][3] = \{B\}$$

$$\text{deoarece } B \rightarrow CC$$

$$T[3][4] = \{S, C\}$$

$$\text{deoarece } S \rightarrow AB, C \rightarrow AB$$

$$T[4][5] = \{A, S\}$$

$$\text{deoarece } A \rightarrow BA, S \rightarrow BC$$

Diagonala 3:

$$T[1][3] = \emptyset$$

$$T[2][4] = \{B\}$$

$$T[3][5] = \{B\}$$

Diagonala 4:

$$T[1][4] = \emptyset$$

$$T[2][5] = \{A, S, C\}$$

Diagonala 5:

$$T[1][5] = \{A, S, C\}.$$

Deoarece:

$$S \in T[1][5],$$

rezultă că:

$$baaba \in L(G).$$

1.3. Parsarea LR(1). Parsarea LR(1) este o metodă *bottom-up* de analiză sintactică, folosită în implementarea compilatoarelor.

Litera **L** indică faptul că intrarea este citită de la stânga la dreapta.

Litera **R** indică faptul că se construiește o derivare dreaptă în ordine inversă.

Simbolul **(1)** indică folosirea unui singur simbol de lookahead.

Definiție 1.7 (Item LR(1)). *Un item LR(1) este de forma:*

$$[A \rightarrow \alpha \bullet \beta, a],$$

unde:

- $A \rightarrow \alpha\beta$ este o producție;
- $\bullet$ marchează poziția curentă în producție;
- a este simbolul de lookahead.

1.3.1. Operațiile Closure și Goto. Closure.

Dacă avem un item:

$$[A \rightarrow \alpha \bullet B\beta, a],$$

atunci trebuie să adăugăm itemi pentru producțiile lui B :

$$[B \rightarrow \bullet \gamma, b],$$

pentru fiecare producție $B \rightarrow \gamma$ și pentru fiecare:

$$b \in \text{FIRST}(\beta a).$$

Goto.

Operația $\text{Goto}(I, X)$ mută punctul peste simbolul X în toate itemele unde acest lucru este posibil, apoi aplică operația Closure.

$$\text{Goto}(I, X) = \text{Closure}(\{[A \rightarrow \alpha X \bullet \beta, a] \mid [A \rightarrow \alpha \bullet X\beta, a] \in I\}).$$

1.3.2. Tabela de parsare. Tabela LR(1) are două componente:

- **Action**, pentru terminale;
- **Goto**, pentru neterminale.

În tabela **Action** putem avea:

- *shift*, dacă trebuie citit următorul simbol;
- *reduce*, dacă trebuie aplicată o producție;
- *accept*, dacă parsarea s-a terminat cu succes;
- *error*, dacă intrarea nu este validă.

Observație 1.8. Dacă într-o celulă a tabelului Action apar două acțiuni diferite, atunci avem un conflict. Conflictele pot fi de tip *shift/reduce* sau *reduce/reduce*. În acest caz, gramatica nu este LR(1).

1.3.3. *Algoritmul de parsare.* Parserul LR(1) menține o stivă care alternează simboluri gramaticale și stări. La fiecare pas:

- (1) se citește starea din vârful stivei;
- (2) se citește simbolul curent de intrare;
- (3) se consultă tabela Action;
- (4) se execută acțiunea corespunzătoare.
 - **Shift:** se pune simbolul curent pe stivă și se avansează în intrare.
 - **Reduce:** se aplică o producție $A \rightarrow \alpha$, se scot de pe stivă simbolurile corespunzătoare lui α , apoi se folosește tabela Goto.
 - **Accept:** cuvântul este acceptat.
 - **Error:** cuvântul este respins.

2. IMPLEMENTARE

2.1. **Reprezentarea gramaticii.** În implementare, o gramatică poate fi reprezentată ca un dicționar. Cheile sunt neterminalele, iar valorile sunt listele de producții.

```
grammar = {
    'S': [('a', 'S', 'b'), ('lambda',)]
}
```

În practică, este important să folosim o reprezentare clară pentru:

- terminale;
- neterminale;
- simbolul de start;
- producția vidă λ ;
- simbolul de final al intrării, de obicei \$.

2.2. **Implementarea transformării în CNF.** Pentru transformarea în CNF este util să implementăm separat fiecare pas.

2.2.1. *Neterminale anulabile.* Primul pas este determinarea neterminalelor care pot genera λ .

```
def find_nullable(grammar):
    nullable = set()
    changed = True

    while changed:
        changed = False

        for A, productions in grammar.items():
            for production in productions:
                if production == [] or all(x in nullable for x in production):
                    if A not in nullable:
                        nullable.add(A)
                        changed = True

    return nullable
```

Observație 2.1. *Ideea este să repetăm procesul până când nu mai descoperim neterminale anulabile noi.*

2.2.2. *Binarizarea producăiilor.* Pentru o producăie lungă, de forma:

$$A \rightarrow X_1 X_2 X_3 X_4,$$

introducem neterminale auxiliare:

$$A \rightarrow X_1 N_1, \quad N_1 \rightarrow X_2 N_2, \quad N_2 \rightarrow X_3 X_4.$$

În implementare, acest pas se face parcurgând producăiile și înlocuind orice producăie cu lungime mai mare decât 2.

Observație 2.2. *Nu este necesar să memorăm semnificația neterminalelor noi. Ele sunt doar simboluri auxiliare introduse pentru a respecta forma CNF.*

2.3. **Implementarea algoritmului CYK.** Pentru CYK construim o matrice triunghiulară:

$$T[i][j].$$

Fiecare celulă conține o mulțime de neterminale.

```
T = [[set() for _ in range(n)] for _ in range(n)]
```

Mai întâi completăm diagonala principală:

```
for i, ch in enumerate(word):
    for A, productions in grammar.items():
        if [ch] in productions:
            T[i][i].add(A)
```

Apoi completăm celulele corespunzătoare subșirurilor mai lungi:

```
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1

        for k in range(i, j):
            # combinam T[i][k] cu T[k+1][j]
            ...
```

La final, verificăm dacă simbolul de start se află în celula corespunzătoare întregului cuvânt.

```
return 'S' in T[0][n - 1]
```

Observație 2.3. *Cele trei bucle imbricate explică de ce algoritmul are complexitate cubică în lungimea cuvântului.*

2.4. **Implementarea parserului LR(1).** Implementarea unui parser LR(1) urmează exact pașii teoretici:

- (1) augmentarea gramaticii;
- (2) calculul mulțimilor FIRST;
- (3) construirea itemilor LR(1);
- (4) implementarea operațiilor Closure și Goto;
- (5) construirea colecției canonice de stări;
- (6) construirea tabelelor Action și Goto;
- (7) parsarea propriu-zisă.

2.4.1. *Itemi LR(1).* Un item LR(1) poate fi reprezentat printr-un obiect sau tuplu:

(stanga, dreapta, pozitie_punct, lookahead)

De exemplu:

$$[S \rightarrow a \bullet Ad, \$]$$

poate fi reprezentat prin:

('S', ('a', 'A', 'd'), 1, '\$')

2.4.2. *Closure*. Funcția `closure` pornește de la o mulțime de itemi și adaugă itemi noi atunci când punctul se află înaintea unui neterminat.

```
def closure(items):
    result = set(items)

    while se_adauga_itemi_noi:
        if punctul_este_inaintea_unui_neterminat:
            adauga_productiile_neterminalului

    return result
```

Observație 2.4. *Lookahead-ul noului item se calculează folosind mulțimea FIRST a secvenței care urmează după neterminat.*

2.4.3. *Goto*. Funcția `goto` mută punctul peste un simbol dat:

```
def goto_state(items, symbol):
    moved = set()

    for item in items:
        if punctul_este_inainte_de(symbol):
            moved.add(item_cu_punctul_mutat)

    return closure(moved)
```

Această funcție este folosită pentru a construi tranzițiile dintre stările automatului LR(1).

2.4.4. *Construirea tabelor*. Pentru fiecare stare:

- dacă punctul este înaintea unui terminal, adăugăm o acțiune de tip `shift`;
- dacă punctul este la finalul unei producții, adăugăm o acțiune de tip `reduce`;
- dacă itemul este $[S' \rightarrow S\bullet, \$]$, adăugăm acțiunea `accept`.

```
if punctul_este_inaintea_unui_terminal:
    ACTION[state, terminal] = shift
```

```
elif punctul_este_la_final:
    ACTION[state, lookahead] = reduce
```

```
elif itemul_este_final:
    ACTION[state, '$'] = accept
```

Pentru neterminale, completăm tabela `Goto`:

```
GOTO[state, nonterminal] = next_state
```

2.4.5. *Parsarea*. Parserul folosește o stivă și intrarea extinsă cu simbolul `$`.

```
stack = [0]
input = list(word) + ['$']
```

La fiecare pas se citește acțiunea corespunzătoare:

```
state = stack[-1]
token = input[position]
action = ACTION[state, token]
```

Dacă acțiunea este `shift`, mutăm simbolul curent pe stivă și avansăm în intrare.

Dacă acțiunea este `reduce`, scoatem de pe stivă simbolurile corespunzătoare părții drepte a producției și aplicăm tabela `Goto`.

Dacă acțiunea este `accept`, cuvântul este acceptat.

Observație 2.5. *Implementarea din laboratorul anterior urmează această structură: definește itemi LR(1), calculează `closure` și `goto`, construiește stările canonice, completează tabelele `ACTION/GOTO` și apoi parsează cu `shift`, `reduce` și `accept`.*

3. EXERCITII

Exercițiu 1. Transformați în CNF gramatica:

$$S \rightarrow aAbB$$

$$A \rightarrow aA \mid \lambda$$

$$B \rightarrow bB \mid \lambda$$

Exercițiu 2. Aplicați algoritmul CYK pentru a decide dacă $w = abab$ aparține limbajului generat de gramatica în CNF:

$$S \rightarrow AB \mid SS$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Exercițiu 3. Pentru gramatica:

$$S \rightarrow aSb \mid \lambda,$$

explicați intuitiv de ce parserul $LR(1)$ acceptă cuvântul **aabb** și respinge cuvântul **aabbb**.